
```
print(math.sqrt(5))
```

We get exactly the same output: 2.23606797749979. We've successfully written our own function that calculates square roots. If you're ever stranded on a desert island with no ability to download Python modules like the `math` module, you can rest assured that you can write functions like `math.sqrt()` on your own, and you can thank the Babylonians for their help in giving us the algorithm for it.

Random Number Generators

So far we've taken chaos and found order within it. Mathematics is good at that, but in this section, we'll consider a quite opposite goal: finding chaos in order. In other words, we're going to look at how to algorithmically create randomness.

There's a constant need for random numbers. Video games depend on randomly selected numbers to keep gamers surprised by game characters' positions and movements. Several of the most powerful machine learning methods (including random forests and neural networks) rely heavily on random selections to function properly. The same goes for powerful statistical methods, like bootstrapping, that use randomness to make a static dataset better resemble the chaotic world. Corporations and research scientists perform A/B tests that rely on randomly assigning subjects to conditions so that the conditions' effects can be properly compared. The list goes on; there's a huge, constant demand for randomness in most technological fields.

The Possibility of Randomness

The only problem with the huge demand for random numbers is that we're not quite certain that they actually exist. Some people believe that the universe is deterministic: that like colliding billiard balls, if something moves, its movement was caused by some other completely traceable movement, which was in turn caused by some other movement, and so on. If the universe behaved like billiard balls on a table, then by knowing the current state of every particle in the universe, we would be able to